MILESTONE 4 REPORT

1. Objective

The objective of Milestone 4 is to integrate the trained YOLOv8-based object detection model with a user-friendly web interface and deploy a functional system capable of performing real-time chest X-ray analysis. This phase focuses on connecting the backend AI model with the frontend dashboard to enable seamless user interaction.

2. Work Done

- 2.1 Backend Development
  - Developed a backend using Flask framework
  - Integrated YOLOv8 trained model (best.pt)
  - Implemented REST API endpoint /predict
  - Enabled image upload and processing
  - Generated detection outputs with bounding boxes
  - Returned structured JSON response including:
    - Prediction (Normal / Abnormal)
    - Confidence score
    - Detected conditions
    - Output image path

- 2.2 Frontend Integration
  - Connected frontend dashboard with backend API
  - Implemented image upload functionality
  - Used Fetch API to send POST requests
  - Displayed:
    - Prediction results
    - Confidence score
    - Processed output image
  - Handled errors such as:
    - Invalid inputs

    - Backend failures

- 2.3 Model Integration
  - Loaded YOLOv8 model using Ultralytics
  - Performed inference on uploaded images
  - Extracted:
    - Class labels
    - Confidence scores
  - Generated annotated output images

3. Tools and Technologies Used

- Programming Language: Python, JavaScript
- Frameworks: Flask, React/HTML Dashboard
- Libraries: Ultralytics YOLOv8, OpenCV, NumPy
- Frontend: HTML, CSS, JavaScript
- Backend: Flask API
- Model: YOLOv8 (Object Detection)
- Platform: Windows

4. Results

- Successfully integrated AI model with web interface
- Achieved real-time prediction capability
- Model outputs displayed correctly in UI
- Final performance (from training):

Metric	Value
Precision	~0.48
Recall	~0.30
mAP@50	~0.336
mAP@50-95	~0.174

5. Challenges Faced

- Dependency conflicts (Python environments, pip issues)
- Integration mismatch (TensorFlow vs YOLO)
- CORS issues between frontend and backend
- File path handling for model weights
- Frontend expecting JSON vs backend returning image

6. Solutions Implemented

- Standardized Python environment using py
- Replaced TensorFlow model with YOLO-based pipeline
- Enabled CORS using flask-cors
- Modified backend to return JSON + image path
- Updated frontend to correctly handle API response

7. Conclusion

Milestone 4 successfully transformed the project into a fully functional AI-powered web application. The system can now accept chest X-ray images from users and provide automated analysis using a trained YOLOv8 model. This milestone bridges the gap between model development and real-world usability.

8. Future Scope

- Improve model accuracy with larger datasets
- Deploy application on cloud (AWS / Render / HuggingFace)
- Add user authentication system
- Store prediction history
- Convert system into mobile application